

Reducing LLM Decision Rights in Agentic AI Systems: A Distillation-Based Design Pattern

Oliver Zhou Lily Li

Agentic AI systems increasingly assign decision authority to large language models (LLMs). This design choice improves scalability and adaptability because the same model can interpret new contexts, select tools, route workflows, and generate action plans without task-specific engineering. However, it also introduces practical concerns about reliability, transparency, reproducibility, governance, and accountability. This paper proposes a design pattern for reducing the decision rights of LLM components without removing the benefits that make them useful. The central idea is to use an LLM more heavily during a learning period, record its decisions and the relevant task context, and then distill recurring decisions into a local deterministic or semi-deterministic machine learning model for the running period. The local model does not need to replace the LLM in all cases. It can decide only when confidence is high, provide recommendations to the LLM, approve or reject LLM outputs, retrieve similar precedents, or participate in a voting mechanism. We describe the mechanism, its relationship to cognitive design patterns, and several initial use cases, including blog article structure selection, cover image theme selection, and economic data insight selection. The paper frames the proposed approach as a practical method for redistributing decision rights inside agentic systems rather than merely reducing inference cost.

Introduction

A significant trend in agentic AI is to give large language models more decision rights. In many recent systems, the LLM is not only a text generator. It interprets the user request,

decomposes the task, selects tools, chooses retrieval sources, writes prompts, evaluates intermediate outputs, and decides whether the workflow should continue, stop, or retry. This architecture is attractive because it turns the LLM into a flexible controller. Instead of designing a separate rule set for every task, developers can rely on the model’s general language understanding, contextual reasoning, and ability to adapt to unfamiliar inputs. Surveys and frameworks on LLM-based autonomous agents describe this central-controller role through memory, planning, profiling, action, and feedback modules (L. Wang et al. 2023).

The same design choice also creates a governance problem. When the LLM receives broad decision rights, a large portion of system behavior becomes stochastic, difficult to reproduce, and difficult to explain at the level required by many production environments. Agentic systems are not merely single model calls; they are multi-step workflows in which reasoning, memory, planning, tool execution, and environment feedback interact over time. This creates interpretability and accountability challenges that cannot be solved by explaining an isolated model output alone (Zhu et al. 2026). It also creates safety and security risks because local vulnerabilities in prompts, tools, memory, retrieval, or orchestration can propagate through the system (Ghosh et al. 2025).

This paper proposes a practical response: reduce the runtime decision rights of LLM components by distilling their previous decisions into local deterministic or semi-deterministic models. The proposal does not reject LLMs. On the contrary, it uses them as teachers during the learning period, when broad adaptability is needed and the system has not yet accumulated enough local experience. After repeated decisions are observed, the system trains a local classifier, ranker, or policy model that can handle recurrent cases with greater transparency and stability. The LLM remains available for uncertain, novel, high-risk, or low-confidence cases.

We use the term *decision rights* to mean the operational authority to determine a system state or workflow transition. Examples include choosing an article structure, selecting a cover image theme, deciding which retrieval source to use, approving whether a generated answer is ready, or choosing which next tool to call. The term does not refer to moral or legal rights of AI systems. It is a design term about how much authority an LLM component has inside a larger agentic architecture.

The core claim is simple: many agentic decisions that initially require LLM judgment later become local, repetitive, and classifiable. Once such regularities are discovered, the system can transfer part of the decision authority from a stochastic LLM to a local model. This transfer can improve reproducibility, auditability, latency, and cost, while preserving scalability through selective deferral back to the LLM.

Background and motivation

Benefits of assigning decision rights to LLMs

LLMs are useful in agentic systems because they provide a general interface between unstructured human goals and structured computational actions. A user may describe a goal in natural language, while the system must translate that goal into workflow steps, tool calls, retrieval operations, and output formats. LLMs are well suited to this role because they can operate across many domains without requiring a separately engineered decision tree for each domain.

This flexibility matters especially when tasks change frequently. In a content production workflow, the same agent may need to decide the structure of a thought-leadership article, the tone of a social post, the theme of a cover image, and the framing of an economic data insight. In a more operational system, the agent may need to route requests among tools, detect when human approval is required, or select the most appropriate prompt strategy. Agentic AI typologies describe this kind of system through dimensions such as autonomy, adaptability, contextualization, operation, and normative alignment (Wissuchek and Zschech 2025).

In the early stage of system development, assigning many decisions to the LLM is often rational. The developer may not yet know which categories matter, which features are predictive, or where the task boundaries should be. The LLM can supply provisional decisions, natural-language rationales, and examples. It can also help the system explore the decision space before a more local representation is available.

Costs of assigning decision rights to LLMs

The cost of broad LLM decision rights is not limited to API fees. The deeper cost is that critical workflow transitions become less stable and less inspectable. The same input may lead to different outputs across model versions, prompts, temperatures, and hidden provider changes. Even when the model provides a persuasive rationale, the rationale may not be a faithful explanation of the internal process that produced the decision. Recent work has warned against treating fluent LLM accounts as full decision explanations without stronger validation (W. Wang 2026).

Agentic systems amplify this problem because a small early decision can affect later retrieval, tool selection, memory updates, and output evaluation. Studies of autonomous agents show that failures can occur in planning, execution, or response generation, and errors can propagate through the feedback loop (Lu, Li, and Huo 2025). As a result, the practical question is not simply whether an LLM is capable. The question is where the system should locate decision authority at each point in the workflow.

Several strands of research already address adjacent parts of this problem. LLM routing work studies how to allocate queries among models with different costs and capabilities (Varangot-Reille et al. 2025; Ong et al. 2024; Chen, Zaharia, and Zou 2023). Knowledge distillation and model compression study how knowledge from larger models can be transferred into smaller models (Buciluă, Caruana, and Niculescu-Mizil 2006; Hinton, Vinyals, and Dean 2015). Cognitive design pattern work emphasizes memory, commitment, reconsideration, and knowledge compilation in agent architectures (Wray, Kirk, and Laird 2025). Human-in-the-loop architectures study how intervention and approval can be decoupled from the core workflow (Cheng and Cheng 2026).

This paper is closest to these areas but differs in its objective. We are not primarily trying to reduce financial cost, although cost reduction may occur. We are trying to optimize the distribution of decision rights between stochastic LLM components and local, auditable components. The key design question is: *when should the LLM decide, and when should a local model decide, approve, retrieve, or escalate?*

Proposed mechanism

The proposed mechanism has two phases: a learning period and a running period.

During the learning period, the LLM is given more decision rights. It observes inputs, makes decisions, provides rationales, and optionally receives feedback from humans or downstream task outcomes. The system records these episodes as local training data. Each record contains the task context, the decision point, the LLM decision, the rationale or structured explanation, the final accepted decision, the features used, and any human override.

During the running period, the system relies less on the LLM. A local model is trained on the accumulated decision records. When a new case arrives, the local model produces a prediction and a confidence score. If confidence is high and the case falls within the model’s allowed scope, the local model can make the decision. If confidence is low, the system defers to the LLM, asks for human approval, or uses a hybrid voting procedure.

The mechanism can be summarized as follows:

1. Identify a repeated decision point in an agentic workflow.
2. Let the LLM decide during the learning period.
3. Log the input context, LLM decision, rationale, and final accepted outcome.
4. Convert the context into structured or vectorized features.
5. Train a local deterministic or semi-deterministic model.
6. Deploy the local model with confidence thresholds, scope constraints, and audit logs.
7. Defer uncertain or novel cases to the LLM or human reviewer.

8. Periodically relearn when the distribution changes or the local model’s error rate increases.

The local model does not need to fully replace the LLM. We distinguish five possible levels of local decision rights:

Mode	Role of the local model	Role of the LLM
Retrieval support	Retrieves similar previous decisions	Makes the final decision
Recommendation	Suggests a likely decision	Accepts, revises, or rejects
Approval gate	Checks whether the LLM decision is consistent with past cases	Generates the candidate decision
Direct decision	Makes the decision above a confidence threshold	Handles deferred cases
Voting member	Votes with the LLM and possibly human rules	Participates in ensemble decision

This graded design is important because the goal is not to eliminate LLMs. The goal is to reduce unnecessary LLM decision rights while preserving adaptability. In many practical settings, the local model should only decide in familiar and low-risk cases. In unfamiliar cases, the LLM remains valuable because its broader reasoning capacity can explore new categories and generate new examples. Over time, those examples become new local memory.

Decision-right thresholding

The decision-right threshold is the policy that determines when the local model is allowed to act. A simple version is:

$$\text{local_decide}(x) = \begin{cases} 1, & \text{if } p(\hat{y} \mid x) \geq \tau \text{ and } x \in S, \\ 0, & \text{otherwise.} \end{cases}$$

where x is the case representation, $\hat{y}$ is the local model’s predicted decision, $p(\hat{y} \mid x)$ is the estimated confidence, τ is the confidence threshold, and S is the approved scope of the local model. Scope constraints prevent the local model from acting outside the distribution it was designed for.

This threshold makes the mechanism tunable. A higher threshold gives fewer decision rights to the local model and more to the LLM. A lower threshold transfers more runtime authority

to the local model. The threshold can also differ by decision type. A cover image theme may allow a lower threshold because the consequence of error is modest. A financial or safety-related decision may require a higher threshold and human approval.

Relearning and reconsideration

A system that reduces LLM decision rights still needs a way to regain adaptability. We therefore include a relearning module. The relearning module monitors distribution shift, confidence decline, override frequency, and performance degradation. When the local model repeatedly defers or receives corrections, the system reopens the learning period for that decision point.

This resembles the cognitive idea of knowledge compilation: repeated reasoning episodes can be converted into more efficient procedural knowledge. It also relates to agent memory architectures, in which past episodes are stored and retrieved to guide future action (Fiorini et al. 2025). In our proposal, memory is not only used to condition the LLM; it is also used to train a local decision component. From a cognitive-architecture perspective, the system localizes part of memory and meta-memory outside the LLM.

Use cases

The first use cases come from content and insight production workflows. They are intentionally modest. The purpose is not to prove that all agentic decisions can be localized, but to demonstrate how the design pattern works on repeated decisions where the input and output space can be defined.

Article structure selection

The first use case is determining the writing structure of a Cognaptus blog article. In the initial workflow, an LLM reads the topic, target audience, source material, intended message, and publishing context. It then selects an article structure, such as explanatory essay, problem-solution, trend analysis, case-based argument, data commentary, or opinion note.

During the learning period, each LLM decision is recorded with the topic, audience, goal, source density, desired tone, length, and final human feedback. After enough examples accumulate, a local classifier is trained to predict the preferred structure. At runtime, the classifier can directly select the structure for common cases. When it is uncertain, it can provide the top two candidate structures to the LLM or ask the LLM to make the decision.

This use case is suitable because the decision is repeated, the output classes are finite, and human feedback is available. It also has a clear quality metric: whether the selected structure leads to an article that is accepted with fewer revisions.

Cover image theme selection

The second use case is determining the cover image theme for a Cognaptus blog article. The LLM initially chooses among themes such as abstract concept, human collaboration, data visualization, market metaphor, technology interface, or industry scene. This decision often depends on the article title, topic, intended emotional tone, audience, and brand style.

A local model can learn from previous LLM and human decisions. For example, articles about abstract organizational strategy may often use metaphorical or abstract images, while data-heavy macroeconomic notes may use chart-based or market-themed images. The local model can then select a theme when the article matches previous patterns. For ambiguous or novel topics, the decision is deferred to the LLM.

This use case is useful because it separates creative generation from decision control. The local model does not need to generate the image. It only selects the theme or approves whether a proposed theme is consistent with brand memory. The generative model can still create candidate images, while the local model controls part of the decision process.

Economic data insight selection

The third use case is selecting the main insight from economic data. Compared with the first two use cases, this decision is more difficult and more consequential. The input may include time series, metadata, market expectations, recent changes, and domain-specific interpretation. The output may be an insight frame, such as trend continuation, structural break, anomaly, policy implication, sector divergence, or risk alert.

For this use case, the local model may not deserve full decision rights. A better role may be recommendation, retrieval, or voting. It can retrieve similar historical cases, classify the likely insight type, and indicate whether the current case resembles known patterns. The LLM can then use this local precedent as a constraint or reference rather than deciding from scratch. This illustrates that reducing LLM decision rights does not always mean direct replacement. Sometimes the appropriate reduction is to give the LLM less unchecked authority by adding local precedent and deterministic review.

Evaluation plan

The empirical evaluation should compare three system variants:

1. **LLM-only decision:** the LLM makes the decision at each decision point.
2. **Local-only decision:** the local model makes the decision whenever it has a prediction.
3. **Hybrid decision-right allocation:** the local model decides only above threshold and defers other cases to the LLM or human reviewer.

The central evaluation object is not only task accuracy. We also evaluate the distribution of decision rights. A system can be accurate but still assign too much authority to the LLM in cases where a simpler local model would be sufficient. Conversely, a system can be cheap but unsafe if it transfers too much authority to a weak local model.

The following metrics can be reported:

Metric	Meaning
Agreement with accepted decision	Whether the model matches the final human-accepted or production-accepted decision
Downstream quality	Whether the resulting article, image theme, or insight requires fewer revisions
LLM decision share	Percentage of cases where the LLM makes the final decision
Local decision share	Percentage of cases where the local model makes the final decision
Deferral rate	Percentage of cases sent back to the LLM or human reviewer
Override rate	Percentage of local decisions later corrected by human or LLM review
Stability	Consistency of decisions across repeated runs on the same input
Interpretability	Ability to inspect features, examples, or rules behind the decision
Cost and latency	Reduction in API calls and response time

The main expected pattern is not that the local model always outperforms the LLM. The expected pattern is that a hybrid system can preserve most accepted decisions while reducing the LLM’s runtime decision share. The threshold should produce a curve: as the confidence

threshold decreases, more decisions shift to the local model, but override risk may increase. The optimal point depends on task risk and organizational preference.

For the two content use cases, evaluation can begin with retrospective logs. We can collect past article topics, LLM recommendations, human-selected structures, image themes, and revision outcomes. A local classifier can be trained on earlier cases and tested on later cases. For the economic insight use case, evaluation should be more conservative. The local model can first be evaluated as a retrieval and recommendation component before being allowed to make direct decisions.

Contributions

This paper makes three contributions.

First, it introduces *decision rights* as a practical design object in agentic AI systems. Instead of asking only whether an LLM can perform a task, the paper asks how much decision authority the LLM should have at each workflow point. This framing is aligned with recent work treating autonomy as a deliberate design choice rather than an automatic consequence of model capability (Feng, McDonald, and Zhang 2025).

Second, it proposes a distillation-based design pattern for reducing LLM decision rights. The pattern uses the LLM as a teacher during the learning period and transfers repeated decisions into a local model during the running period. This extends the spirit of model distillation and compression from computational efficiency to authority allocation. The central objective is not merely to use a cheaper model, but to build a system in which repeated decisions become more stable, inspectable, and governable.

Third, it connects agentic AI governance with local memory and knowledge compilation. Retrieval-augmented generation and memory modules are often understood as ways to improve LLM output. We interpret them from another perspective: local memory can also reduce the need for stochastic runtime decisions. When memory is structured into a classifier, policy, or approval gate, it becomes a local decision component rather than only context for the LLM.

Limitations

This paper is an initial design proposal. It does not claim that local models should replace LLMs in all agentic decisions. Some decisions are rare, open-ended, creative, or highly

context-dependent. In those cases, the LLM may remain the most appropriate decision maker, and the local model should only retrieve precedents or provide checks.

The proposal also depends on the quality of the logged decisions. If the LLM’s early decisions are poor, biased, or misaligned with human preferences, distilling them into a local model may reproduce those errors. Human feedback and final accepted outcomes should therefore be included in the training data whenever possible. The local model should learn from accepted decisions, not only from raw LLM outputs.

Another limitation is distribution shift. A local model trained on past decisions may become outdated when the brand strategy, audience, market environment, or task definition changes. This is why the relearning module is not optional. Deferral, override, and performance metrics should be monitored continuously.

Finally, the term decision rights requires careful use. It is useful as a system-design concept, but it should not imply that the LLM has independent agency outside the architecture. Decision rights are assigned by system designers through prompts, tools, thresholds, interfaces, and governance policies.

Conclusion

Agentic AI systems gain scalability and adaptability by giving LLMs broad decision rights. However, broad LLM decision rights also create practical problems of stability, transparency, reproducibility, and accountability. This paper proposes a design pattern that uses LLMs heavily when the system is learning, then reduces their runtime decision authority by distilling repeated decisions into local deterministic or semi-deterministic models.

The proposed mechanism does not remove the LLM. It changes its role. The LLM becomes a teacher, fallback decision maker, explorer of novel cases, and participant in hybrid workflows. The local model becomes a memory-based decision component that handles familiar cases, provides approvals, retrieves precedents, or participates in voting. The result is a more balanced agentic system in which adaptability and governance are not treated as opposites.

Future work should implement the proposed mechanism in the three use cases described above and report threshold curves showing the trade-off between decision-right reduction, quality, deferral, override risk, cost, and latency. The broader research agenda is to treat decision authority as a configurable property of agentic systems and to design architectures that allocate this authority deliberately rather than implicitly giving it to the most general model.

References

- Buciluă, Cristian, Rich Caruana, and Alexandru Niculescu-Mizil. 2006. “Model Compression.” *Proceedings of the 12th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 535–41. <https://doi.org/10.1145/1150402.1150464>.
- Chen, Lingjiao, Matei Zaharia, and James Zou. 2023. “FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance.” <https://arxiv.org/abs/2305.05176v1>.
- Cheng, Edward, and Jeshua Cheng. 2026. “A Decoupled Human-in-the-Loop System for Controlled Autonomy in Agentic Workflows.” <https://arxiv.org/abs/2604.23049v1>.
- Feng, K. J. Kevin, David W. McDonald, and Amy X. Zhang. 2025. “Levels of Autonomy for AI Agents.” <https://arxiv.org/abs/2506.12469v2>.
- Fiorini, Sandro Rama, Leonardo G. Azevedo, Raphael M. Thiago, Valesca M. de Sousa, Anton B. Labate, and Viviane Torres da Silva. 2025. “Episodic Memory in Agentic Frameworks: Suggesting Next Tasks.” <https://arxiv.org/abs/2511.17775v1>.
- Ghosh, Shaona, Barnaby Simkin, Kyriacos Shiarlis, Soumili Nandi, Dan Zhao, Matthew Fiedler, Julia Bazinska, et al. 2025. “A Safety and Security Framework for Real-World Agentic Systems.” <https://arxiv.org/abs/2511.21990v1>.
- Hinton, Geoffrey, Oriol Vinyals, and Jeff Dean. 2015. “Distilling the Knowledge in a Neural Network.” <https://arxiv.org/abs/1503.02531v1>.
- Lu, Ruofan, Yichen Li, and Yintong Huo. 2025. “Exploring Autonomous Agents: A Closer Look at Why They Fail When Completing Tasks.” <https://arxiv.org/abs/2508.13143v1>.
- Ong, Isaac, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M Waleed Kadous, and Ion Stoica. 2024. “RouteLLM: Learning to Route LLMs with Preference Data.” <https://arxiv.org/abs/2406.18665v4>.
- Varangot-Reille, Clovis, Christophe Bouvard, Antoine Gourru, Mathieu Ciancone, Marion Schaeffer, and François Jacquenet. 2025. “Doing More with Less: A Survey on Routing Strategies for Resource Optimisation in Large Language Model-Based Systems.” <https://arxiv.org/abs/2502.00409v3>.
- Wang, Lei, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, et al. 2023. “A Survey on Large Language Model Based Autonomous Agents.” <https://doi.org/10.1007/s11704-024-40231-1>.
- Wang, Wenshuo. 2026. “LLMs Should Not yet Be Credited with Decision Explanation.” <https://arxiv.org/abs/2605.01164v1>.
- Wissuchek, Christopher, and Patrick Zschech. 2025. “Exploring Agentic Artificial Intelligence Systems: Towards a Typological Framework.” <https://arxiv.org/abs/2508.00844v1>.
- Wray, Robert E., James R. Kirk, and John E. Laird. 2025. “Applying Cognitive Design Patterns to General LLM Agents.” <https://arxiv.org/abs/2505.07087v2>.
- Zhu, Judy, Dhari Gandhi, Himanshu Joshi, Ahmad Rezaie Mianroodi, Sedef Akinli Kocak, and Dhanesh Ramachandran. 2026. “Interpreting Agentic Systems: Beyond Model Ex-

planations to System-Level Accountability.” <https://arxiv.org/abs/2601.17168v1>.